

Vortragsskript zur Präsentation

Das Skript ist eng an deine Präsentation und die Inhalte deiner Facharbeit angepasst. Bei normalem Sprechtempo dauert der Vortrag ungefähr **8 bis 10 Minuten**.

Titelfolie

„Guten Tag zusammen,

mein Name ist Firas Massoussi und ich studiere Cyber Security and Privacy.

In meinem Abschlussvortrag beschäftige ich mich mit dem Thema **automatisierte Sicherheitstests in CI/CD-Pipelines**. Dabei geht es insbesondere um DevSecOps als Ansatz für eine sichere Softwareentwicklung.

Ich möchte heute zeigen, warum Sicherheitsprüfungen nicht erst am Ende der Entwicklung stattfinden sollten, welche automatisierten Testarten es gibt und an welchen Stellen einer CI/CD-Pipeline diese sinnvoll eingesetzt werden können.

Außerdem werde ich auf die wichtigsten Vorteile und Herausforderungen von DevSecOps eingehen.“

Folie 2 – Motivation: Warum das Thema wichtig ist

„Zuerst möchte ich erklären, warum dieses Thema überhaupt wichtig ist.

Software wird heute immer schneller entwickelt und veröffentlicht. Viele Unternehmen arbeiten mit agilen Methoden, DevOps und CI/CD-Pipelines. Dadurch können Änderungen in kurzen Abständen gebaut, getestet und bereitgestellt werden.

Diese Geschwindigkeit bringt aber auch ein Sicherheitsproblem mit sich. Wenn Sicherheit erst kurz vor der Veröffentlichung geprüft wird, werden Schwachstellen möglicherweise sehr spät entdeckt.

Zusätzlich besteht moderne Software häufig aus vielen externen Komponenten. Dazu gehören Open-Source-Bibliotheken, Frameworks und Container-Images. Auch versehentlich veröffentlichte Zugangsdaten wie API-Keys oder Passwörter können ein großes Risiko darstellen.

Die zentrale Idee lautet deshalb: Bei einer schnellen Softwareentwicklung darf Sicherheit kein einzelner Schritt am Ende sein. Sie muss frühzeitig, regelmäßig und möglichst automatisiert in den Entwicklungsprozess integriert werden.“

Folie 3 – Forschungsfrage

„Daraus ergibt sich die Forschungsfrage meiner Arbeit:

Wie können automatisierte Sicherheitstests in CI/CD-Pipelines dazu beitragen, Sicherheitsprobleme frühzeitig im Softwareentwicklungsprozess zu erkennen, und welche Vorteile sowie Herausforderungen ergeben sich daraus für das Software Engineering?

Um diese Frage zu beantworten, habe ich drei Bereiche betrachtet.

Erstens: Was ist DevSecOps und wie unterscheidet es sich von DevOps?

Zweitens: Welche automatisierten Sicherheitstests gibt es und an welcher Stelle der Pipeline sollten sie eingesetzt werden?

Und drittens: Welche Vorteile bringt dieser Ansatz und wo liegen seine Grenzen?

Bei meiner Arbeit handelt es sich um eine Literaturlarbeit. Ich habe dafür wissenschaftliche Veröffentlichungen und anerkannte fachliche Quellen wie NIST und OWASP ausgewertet und miteinander verglichen.“

Folie 4 – Von DevOps zu DevSecOps: Shift Left

„Um DevSecOps zu verstehen, muss zunächst der Begriff DevOps betrachtet werden.

DevOps verbindet die Bereiche Entwicklung und Betrieb. Ziel ist es, Software schneller und zuverlässiger bereitzustellen. Dazu werden beispielsweise Builds, Tests und Deployments automatisiert.

Das Problem ist jedoch, dass Sicherheit bei einem klassischen DevOps-Prozess teilweise weiterhin getrennt betrachtet wird.

DevSecOps erweitert DevOps deshalb um den Sicherheitsaspekt. Security wird nicht erst nachträglich ergänzt, sondern soll von Anfang an Teil des gesamten Prozesses sein.

Dabei handelt es sich nicht nur um einen technischen Ansatz. Auch die Zusammenarbeit und die Verantwortlichkeiten innerhalb eines Teams spielen eine wichtige Rolle. Entwicklung, Betrieb und Security sollen gemeinsam für die Sicherheit verantwortlich sein.

In diesem Zusammenhang wird häufig der Begriff **Shift Left Security** verwendet. Das bedeutet, dass Sicherheitsprüfungen weiter nach links, also an einen früheren Punkt im Entwicklungsprozess, verschoben werden.

Fehler sollen möglichst nah an ihrer Entstehung erkannt werden und nicht erst kurz vor der Veröffentlichung.“

Folie 5 – Die DevSecOps-Pipeline

„Auf dieser Folie ist vereinfacht dargestellt, an welchen Stellen einer CI/CD-Pipeline Sicherheitstests eingesetzt werden können.

Bereits beim Commit oder Pull Request können Secret Scanning und SAST ausgeführt werden. Dabei wird geprüft, ob Zugangsdaten im Repository enthalten sind oder ob der Quellcode möglicherweise unsichere Muster enthält.

Während des Builds können externe Bibliotheken und Abhängigkeiten mit Software Composition Analysis, kurz SCA, überprüft werden.

In der Testphase wird die laufende Anwendung mit DAST getestet. Ein mögliches Werkzeug dafür ist beispielsweise OWASP ZAP.

Vor der Veröffentlichung eines Container-Images kann ein Container Scan durchgeführt werden. Dabei wird geprüft, ob das Image bekannte Schwachstellen enthält.

Nach dem Deployment bleiben Monitoring und regelmäßige Überprüfungen wichtig.

Ein weiterer wichtiger Punkt ist, dass auch die Pipeline selbst geschützt werden muss. Sie besitzt häufig Zugriff auf den Quellcode, Zugangsdaten, Build-Artefakte und Deployment-Systeme. Dadurch ist sie ebenfalls ein mögliches Angriffsziel.“

Folie 6 – Fünf automatisierte Testarten

„In meiner Arbeit konzentriere ich mich auf fünf automatisierte Testarten.

Die erste Testart ist **SAST**, also Static Application Security Testing. Dabei wird der Quellcode untersucht, ohne dass die Anwendung ausgeführt werden muss. SAST kann beispielsweise auf unsichere Funktionen oder mögliche Programmierfehler hinweisen.

Die zweite Testart ist **DAST**, also Dynamic Application Security Testing. Hier wird eine bereits laufende Anwendung von außen getestet. Das Werkzeug verhält sich dabei teilweise ähnlich wie ein Angreifer, der unterschiedliche Eingaben und Anfragen ausprobiert.

Die dritte Testart ist **SCA**, also Software Composition Analysis. Dabei werden externe Bibliotheken und Pakete auf bekannte Schwachstellen untersucht.

Secret Scanning sucht nach Zugangsdaten wie API-Keys, Passwörtern oder Tokens im Code und in der Git-Historie.

Container Scanning untersucht schließlich Container-Images und die darin enthaltenen Pakete.

Wichtig ist, dass diese Testarten unterschiedliche Bereiche prüfen. Sie ersetzen sich deshalb nicht gegenseitig, sondern ergänzen sich.“

Folie 7 – Vergleich: Wer prüft was und wann?

„Diese Tabelle vergleicht die fünf Testarten noch einmal nach ihrer Pipeline-Phase, ihrem Prüfobjekt und ihren typischen Funden.

SAST kann bereits beim Commit, Pull Request oder Build eingesetzt werden und untersucht den Quellcode.

DAST benötigt dagegen eine laufende Anwendung. Deshalb wird es normalerweise erst in einer Testumgebung nach dem Build ausgeführt.

SCA wird während des Builds eingesetzt, weil dort die verwendeten Bibliotheken und Pakete bekannt sind.

Secret Scanning sollte möglichst früh erfolgen, beispielsweise beim Commit oder Pull Request. Wird ein Secret erst später entdeckt, reicht es nicht aus, es einfach aus dem Code zu löschen. Es muss normalerweise auch gesperrt oder ersetzt werden.

Container Scanning findet vor dem Deployment statt und untersucht das fertige Container-Image.

Die wichtigste Aussage der Tabelle ist: Kein einzelner Test kann alle Sicherheitsrisiken erkennen. Erst durch die Kombination verschiedener Prüfungen entsteht eine umfassendere Sicherheitskontrolle.“

Folie 8 – Vorteile von DevSecOps

„DevSecOps bietet mehrere Vorteile.

Der erste Vorteil ist die frühere Erkennung von Sicherheitsproblemen. Wenn ein Fehler bereits beim Commit oder Pull Request gefunden wird, kann er direkt von den Entwicklerinnen und Entwicklern untersucht werden.

Ein weiterer Vorteil besteht in der Regelmäßigkeit. Automatisierte Tests können bei jeder Änderung ausgeführt werden und nicht nur einmal am Ende eines Projektes.

DevSecOps kann außerdem die Zusammenarbeit verbessern. Entwicklung, Betrieb und Security müssen gemeinsame Regeln festlegen und zusammen entscheiden, welche Sicherheitsrisiken akzeptabel sind und welche Probleme sofort behoben werden müssen.

Ein weiterer Punkt ist die Nachvollziehbarkeit. Die Pipeline erzeugt Berichte und Testergebnisse. Dadurch lässt sich später nachvollziehen, welche Prüfungen durchgeführt wurden und welche Schwachstellen gefunden wurden.

Solche Informationen können auch für interne Kontrollen, Audits und die Qualitätssicherung hilfreich sein.“

Folie 9 – Herausforderungen von DevSecOps

„Trotz dieser Vorteile gibt es auch mehrere Herausforderungen.

Eine Herausforderung sind Fehllalarme, also sogenannte False Positives.

Sicherheitstools können Meldungen erzeugen, obwohl kein tatsächlich ausnutzbares Problem vorliegt. Wenn zu viele Warnungen entstehen, besteht die Gefahr, dass wichtige Funde übersehen werden.

Auch die Einführung verursacht Aufwand. Die Pipeline muss angepasst werden, Werkzeuge müssen ausgewählt und eingerichtet werden und das Team muss Regeln definieren. Beispielsweise muss entschieden werden, bei welchen Schwachstellen ein Build gestoppt wird.

Eine weitere Herausforderung ist fehlendes Security-Wissen. Ein Werkzeug kann eine Warnung anzeigen, aber das Team muss verstehen, was diese Meldung bedeutet und wie kritisch sie tatsächlich ist.

Außerdem darf man sich nicht zu stark auf die Tools verlassen. Automatisierte Sicherheitstests ersetzen keine sichere Architektur, keine Bedrohungsmodelle und keine menschliche Bewertung.

DevSecOps bedeutet deshalb nicht, einfach möglichst viele Tools einzubauen. Die Ergebnisse müssen auch bewertet, priorisiert und bearbeitet werden.“

Folie 10 – Fazit und Ausblick

„Damit komme ich zum Fazit.

Automatisierte Sicherheitstests können Sicherheitsprobleme früher sichtbar machen, wenn sie sinnvoll über die verschiedenen Phasen einer CI/CD-Pipeline verteilt werden.

SAST und Secret Scanning eignen sich besonders für den frühen Entwicklungsprozess. Dependency Scanning passt in die Build-Phase. DAST untersucht die laufende Anwendung und Container Scanning wird vor dem Deployment eingesetzt.

Die Forschungsfrage lässt sich daher folgendermaßen beantworten:

Automatisierte Sicherheitstests unterstützen die frühe und regelmäßige Erkennung von Sicherheitsproblemen. Ihr tatsächlicher Nutzen entsteht aber nicht allein durch die eingesetzten Werkzeuge. Entscheidend ist das Zusammenspiel von Automatisierung, klaren Prozessen, geschützter Infrastruktur und guter Zusammenarbeit zwischen den beteiligten Personen.

Eine Grenze meiner Arbeit besteht darin, dass es sich um eine Literaturarbeit handelt. Ich habe keine eigene Pipeline praktisch aufgebaut und keine Werkzeuge selbst miteinander verglichen.

Ein möglicher nächster Schritt wäre deshalb, eine Beispiel-Pipeline zu implementieren und dabei zu untersuchen, welche Schwachstellen erkannt werden, wie viele Fehlalarme entstehen und wie stark sich die Laufzeit der Pipeline verändert.

Vielen Dank für Ihre Aufmerksamkeit. Ich freue mich auf Ihre Fragen